



International University, Vietnam National University – HCMC

School of Biomedical Engineering



MEDICAL INSTRUMENTATIONS

FINAL PROJECT REPORT

Tele-Stethoscope

Instructor: *Ph.D Le Ngoc Bich*

Name: Vĩnh Bảo Phúc Hưng	BEBEIU20204
Đỗ Vy Ngọc	BEBEIU20033
Nguyễn Hữu Tường	BEBEIU20047

Semester II, 2021-2022

Table of Contents

LIST OF FIGURES.....	2
LIST OF TABLES.....	ERROR! BOOKMARK NOT DEFINED.
1. INTRODUCTION:.....	3
1.1. MOTIVATION:.....	4
1.2. OBJECTIVE AND INITIAL REQUIREMENTS:.....	4
1.3. RELATED WORKS:.....	5
1.4. IDEA INTRODUCTION.....	5
1.5. DESIGN SPECIFICATIONS:.....	6
2. MATERIALS & METHODS:.....	7
2.1. MATERIALS:.....	7
2.2. BLOCK DIAGRAM:.....	7
2.3. ELECTRICAL WIRING/SCHEMATICS:.....	8
2.4. MECHANICAL DESIGN:.....	9
2.5. FIRMWARE & SOFTWARE:.....	11
2.6. OPERATING PROCEDURE.....	26
3. TESTING METHODOLOGY:.....	28
4. RESULTS & DISCUSSIONS:.....	29
5. COST ANALYSIS:.....	31
6. CONCLUSION & LIMITATION & FUTURE WORKS.....	32
7. TASK DIVISION AND TIMELINE:.....	33
8. REFERENCES:.....	34

List of Figures

Figure 1. LungPass device - the initial design idea.....	5
Figure 2. Telestethoscope's Block Diagram.....	8
Figure 3. The schematic of tele-stethoscope.....	8
Figure 4. Mechanical design and Component placing by 2D drawing.....	9
Figure 5: Box design 2D drawing.....	10
Figure 6: Fusion 360.....	11
Figure 7: Arduino IDE.....	11
Figure 8. The rendered view of the design comparing to the actual product.....	11
Figure 9. The device in use.....	26
Figure 10. Connecting to Wi-Fi.....	27
Figure 11. Receiving email. The email subject and content can be edited inside the code. (a) Two mails send to the patient. (b) Doctor only receive the mail with .wav file attachment. (c) Mail with web server address.....	27
Figure 12. Downloading the files via a local network by copying the address from Figure 12(c).....	28
Figure 13. Operating procedure flowchart.....	28

Figure 14: Telestethoscope's overall appearance.....	29
Figure 15: Device's disassembly. From left to right: Main compartment with sound-proofing layer on top, sound-proofing layer removed and battery compartment..	29
Figure 16: Placement of components inside main chamber. A small rubber earpiece on the left corner connects the stethoscope and microphone.	29
Figure 17: Telestethoscope's dimension.	30
Figure 18. The plot of the received 15-second .wav file showed the noises from the recording. This is our demonstration of a member's heartbeat.....	31
Figure 19. Task division table	33

List of Tables

Table 1. Design specifications table	6
Table 2. Where to get the components	7
Table 3. Payment Table	31

1. Introduction:

1.1. Motivation:

In our current situation, to better serve the control of the Covid-19 epidemic, improve social life, the need for the use of medical services and telemedicine becomes increasingly practical. It is aimed at useful and convenient medical devices that can be used by patients to monitor their health at home. Our product assesses the health status of each patient via their cardiopulmonary sounds. This is similar to how a doctor can assess the condition of a patient through a stethoscope. Therefore, we want our product to mimic the working principle of the stethoscope, but wirelessly and remotely. It can contribute towards our battle against the Coronavirus since it is difficult for the doctor to come in direct contact with a potential carrier to assess their health status. With this device, indirect assessment (or part of it) can be done. Furthermore, when used together with other telemedicine devices, it gives us a step closer to the IU-BME vision of bringing the hospital to the patient's home.

1.2. Objective and Initial requirements:

The objective of this project focuses on the data gathering, more specifically recording the sound from the cardiopulmonary system, and the transmission of audio files between the microcontroller and computer via Wi-Fi to the doctor and patient. Data processing is out of our scope, so it is not covered in this project. Although further filtering and analysis of the audio files is possible and rather easy if performed separately from the collecting phase.

For the current product, our main goals are:

1. To record the audio file since that will give the doctor more information to work with.
2. Transfer the file over the largest distance possible to allow usage at rural areas
3. To simplify the operating procedure so that the patient can use it without any difficulties.
4. To make it as small and affordable as possible

1.3. Related Works:

Our initial idea was to make a compactible design that can be hold by hand. In the market, there has already been some commercial design that inspired us to continue our project. For example, the LungPass device is a device that helps frontline healthcare workers to detect lung sounds indicative of serious respiratory conditions. Its price is approximately 1.300.000VND. Lung Pass design and mechanism also works similar to our project. Thus, we use the device as a reference. Our vision is to record and send the record to qualified doctors to track changes in a patient's lung sound over time.



Figure 1. LungPass device - the initial design idea

1.4. Idea introduction

We will apply the knowledge learned in 3D design, programming and electrical engineering to operate machine units, assemble the boards, as well as the accessories and components that make up the machine.

Our idea is to use a stethoscope chest piece placed on a patient's chest or back to amplify the sound from the lungs, then record them with a microphone into an audio file and send the audio file via Wi-Fi from ESP32 to a computer for assessment and analysis.

For the 3D design, we will draw with Fusion 360 since IU gives us a free license for it. The design should be easy to assemble, operate and preferably small enough to be handheld.

1.5. Design specifications:

Table 1. Design specifications table

1	Target market	Patients with poor cardiopulmonary conditions and their doctors.
2	Performance	The device can record lung sound and send the audio data to the doctors.
3	Accuracy and reliability	The recording is audible and not noisy. Doctors can distinguish between normal and abnormal lung sound. Same results are observed for the same person.
4	Target cost and the availability of components	Lower than 1.300.000 VND. The choices of components will be mainly based on the availability of the component and of their online material
5	Weight and size	Less than 500 g. The size is portable and can be held with one hand comfortably.
6	Life expectancy	Can be used more than 5 times per one charge.
7	Power supply	Either USB Type C adapter or battery. The battery is lithium polymer with 3.7 V and 500 mAh. The adapter will be used when the battery is low or no battery available.
8	Working range	Result can be sent to both local and different Wi-Fi network.
9	Aesthetic appearance	The device will be designed with aesthetics in mind. The shapes follow with smooth line and curve. The color is white to promote cleanness and high-quality in healthcare setting.
10	Manufacturing processes	The device will be manufactured as a prototype, including the use of CAD program. The equipment required are 3D printing machine and hand drill to make some adjustment.
11	Safety	No hazards to the users and the environment.
12	Installation and operation	No installation needed. The operation is easily understandable and user-friendly. There are sounds to announce the performance of each process for debugging.

2. Materials & Methods:

2.1. Materials:

Table 2. Where to get the components

Component name	Where to get
ESP32 – WROOM32	Old projects
INMP441	Hshop
Stethoscope chest piece	Old stethoscope
Charging/boost circuit	ThegioiIC
Lithium-polymer battery	ThegioiIC
Button	Nhật Tảo market
LED	Old projects
Buzzer	Nhật Tảo market
Sound-proofing material	Nhật Tảo market
3D material: ABS	Sponsor

2.2. Block diagram:

In our circuit, to keep the design as compact as possible, we are using a lithium-polymer battery for power supply. Along with the battery, we also use a boost circuit to create a 5V current for the ESP. A button is used to power the ESP and turn the device ON/OFF. Since the battery is rechargeable, there is also an adapter connected with the circuit that can connect with the power source to charge the battery.

In use, the stethoscope is placed on the patient's back or chest to amplify the sound from his/her lung and heart. The sound is then recorded by the INMP441 microphone and saved into the ESP built-in memory (SPIFFS), which can then be transferred to the patient and doctor via email.

An LED and buzzer act as the signaling components that will give the user an insight into the status of the ESP or the procedure. The mechanism of the two components will be explained in the next part of this report.

The block diagram of the device is shown below:

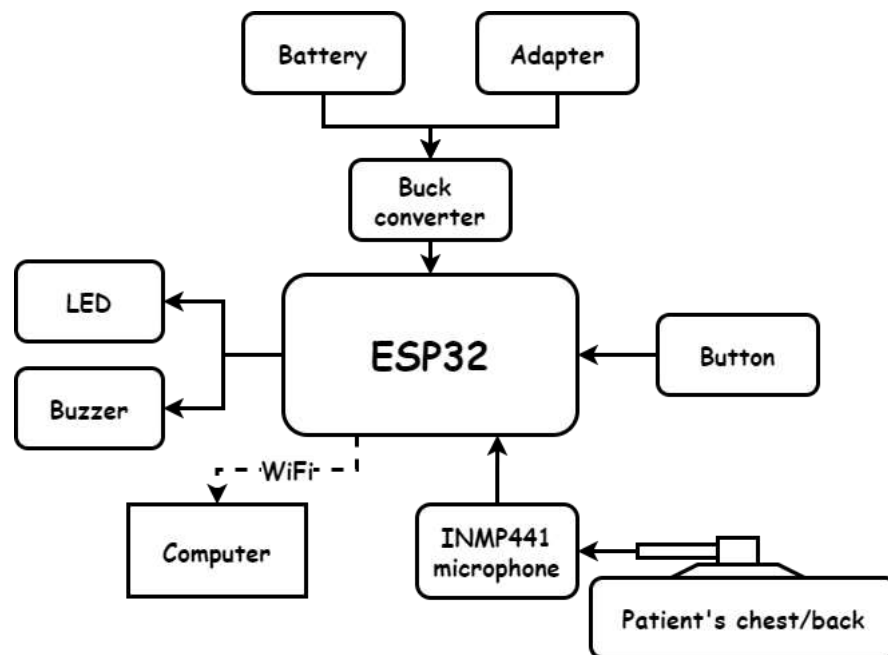


Figure 2. Telestethoscope's Block Diagram

2.3. Electrical Wiring/Schematics:

In our schematic, the ESP32 is the center of control. For power supply, the lithium battery is connected to the boost converter to create a 5V current. From the boost converter, two wires are directly connected to the 5V and GND pins. The two signaling components, the led and buzzer are also linked with the battery and ESP32 via the two wires. On the other side of the schematic, one leg of the button is connected to the GND pins of the ESP32 and the microphone. Other pins of the INMP441 are connected to different pins of the center. The illustration of the schematic is shown in the below figure.

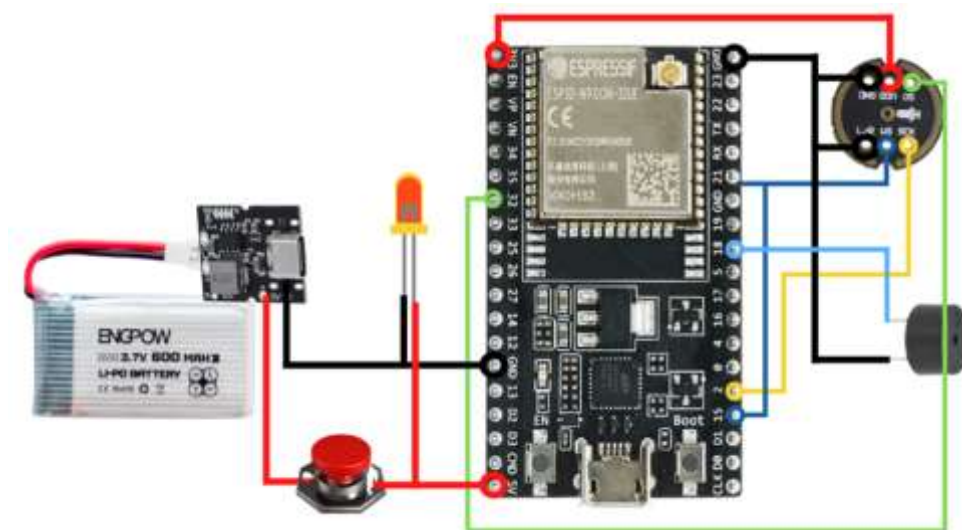


Figure 3. The schematic of tele-stethoscope

2.4. Mechanical Design:

This design is created with the help of Fusion, a 3D CAD program.

There are two hinges on the side of the device. This creates the sense of conform while holding the device. The device is separated into two chambers, the power chamber and the component chamber, so that the patient cannot accidentally access the electrical components but can charge the battery if needed. Some structures are used to fix the components in place.

The 0.5 tolerance is set for every design that requires joining parts together.

The 3D design and the component placement can be seen below:

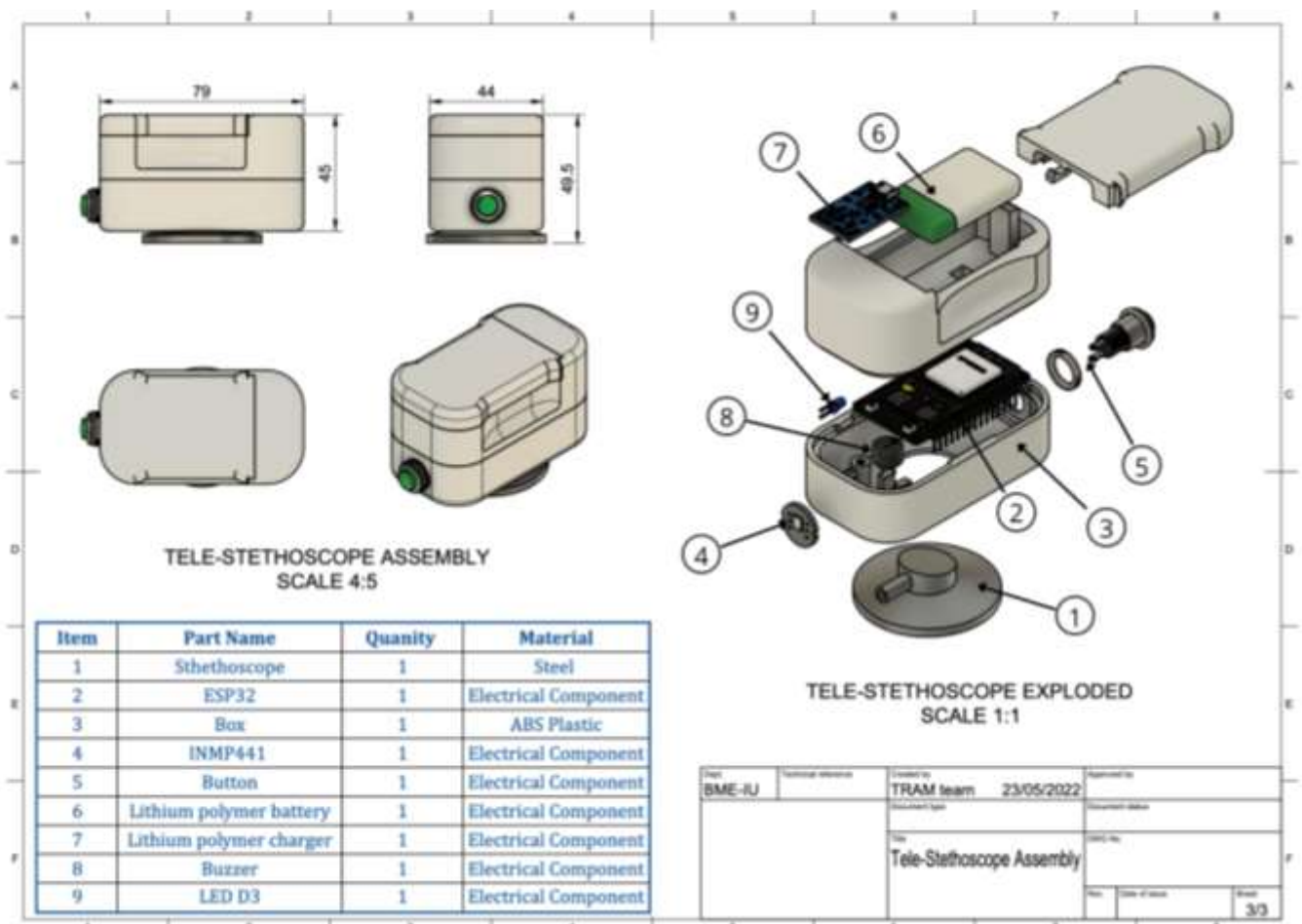


Figure 4. Mechanical design and Component placing by 2D drawing.

Battery compartment:

The battery compartment holds one lithium polymer battery, the power source for the device, and a battery charger with USB type C connection. The battery compartment is closed by the assembly of the sliding lid.

The sliding lid is designed with a Cantilever snap fit with a taper beam at one end. The base of the Cantilever is fillet to help distribute stress, making the connection stronger. The protrusion deflects upon insertion and bends back to the hole inside the compartment. This snap fit allows both assembly and disassembly.

The snap-fit cantilever was built up from the bed horizontally. The build direction was considered thoroughly to avoid weakness due to the anisotropic nature of 3D printing.

The bed of the compartment has a small hole for wiring the battery with electrical components inside the main chamber.

Main compartment:

This compartment is equipped with four 3.5-mm threads for mounting ESP32 device, a circular slot holding the IMNP441 microphones, and two openings, one for push button attachment of 10 mm diameter, and the other for the stethoscope, which has 23.3 mm diameter. There is a small snap-fit joint to fix the stethoscope beam in place and three snap-on fits on the side help connect the two compartments together.

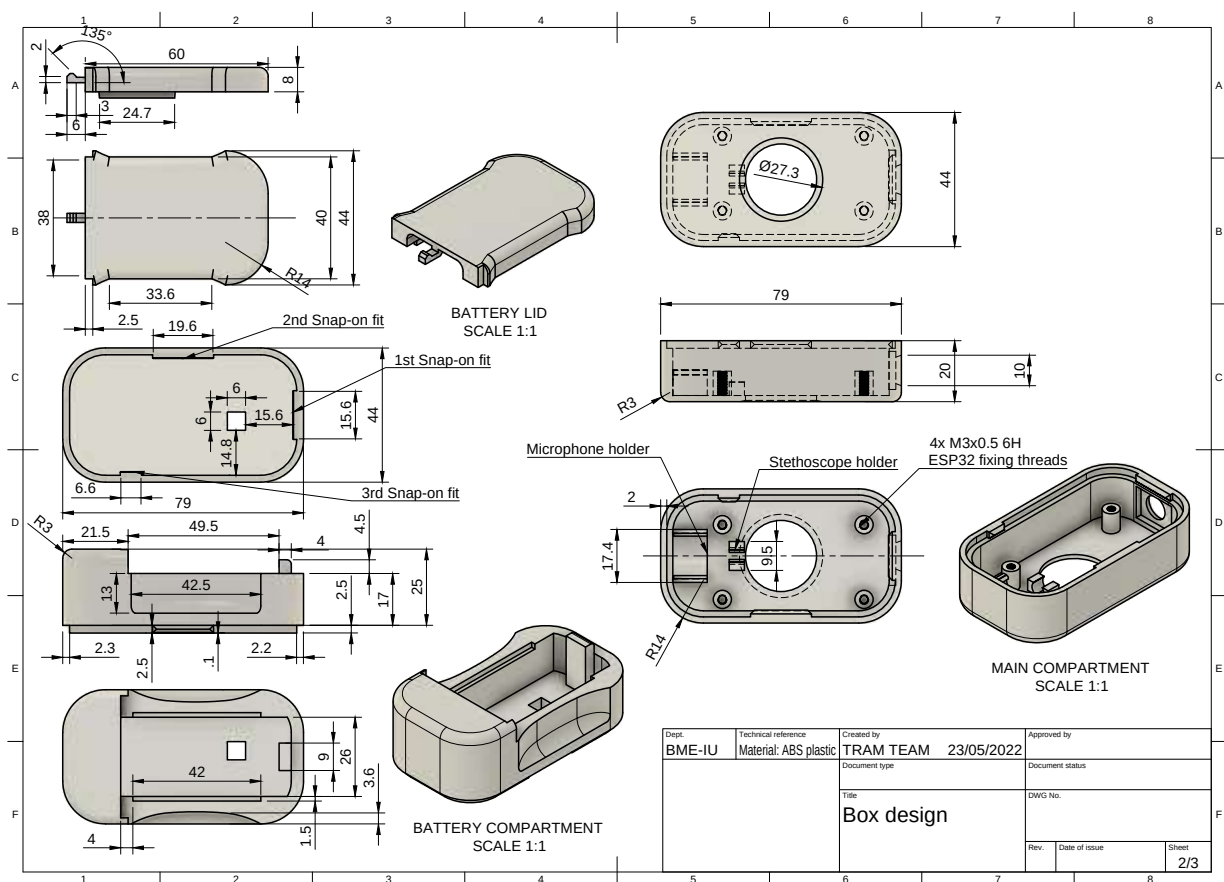


Figure 5: Box design 2D drawing.

2.5. Firmware & Software:



Figure 6: Fusion 360



Figure 7: Arduino IDE

To carry out the project, we used two applications to aid the progress, Autodesk Fusion 360 and Arduino IDE respectively.

Our plan was to make a device that is small and compatible enough to meet the requirements for telemedicine. Fusion 360 is a cloud-based 3D modeling, CAD, CAM, CAE, and PCB software platform for product design and manufacturing. With fusion 360, we managed to create a 3D design for the product. The design enables our team to review the feasibility of our draft and visualize how components should be arranged to maximize product space.



Figure 8. The rendered view of the design comparing to the actual product.

Arduino Software (IDE) is an open source that enables writing and uploading code to the board. In our project, IDE is the main platform to implement code to the ESP32.

Our initial requirements for the coding part are:

- Wi-Fi configuration for ESP32 wirelessly

- Being able to record sound using IMNP441 microphone.
- Store the sound file on ESP32 chip memory.
- Download the sound file via a webserver
- Send the sound file to recipient's emails

To fulfill these requirements, we had to combine codes from three different sources ("ESP32 Send Emails using SMTP Server: HTML, Text, Attachments (Arduino IDE) | Random Nerd Tutorials", 2022), ("ESP32 WiFiManager - Easy WiFi Provisioning", n.d.), (That Project, 2021). This process was time-consuming, done by trial and error, and required general understanding of the libraries used as well as the meaning of certain coding blocks. Our code is shown below.

```
#include <driver/i2s.h>
#include <SPIFFS.h>

#include <WebServer.h>
#include <ESPmDNS.h>

#include <WiFiManager.h>
#include <ESP_Mail_Client.h>

/* The sender sign in credentials */
#define AUTHOR_EMAIL "vn.dovyngoc@gmail.com"
#define AUTHOR_PASSWORD "bgffqbunuymbajcp"
#define SMTP_HOST "smtp.gmail.com"
#define SMTP_PORT 465

/* Recipient's email*/
#define USER_EMAIL "dovyngoc2002@gmail.com"
#define RECIPIENT_EMAIL1 "hungvinh7933@gmail.com"
#define RECIPIENT_EMAIL2 "bebeiu20047@student.hcmiu.edu.vn"

/*IMNP441 pin port*/
#define I2S_WS 15
#define I2S_SD 32
#define I2S_SCK 2
#define I2S_PORT I2S_NUM_0
#define I2S_SAMPLE_RATE (16000)
#define I2S_SAMPLE_BITS (16)
#define I2S_READ_LEN (16 * 1024)
#define RECORD_TIME (15) //Recording time
#define I2S_CHANNEL_NUM (1)
#define FLASH_RECORD_SIZE (I2S_CHANNEL_NUM * I2S_SAMPLE_RATE * I2S_SAMPLE_BITS / 8 * RECORD_TIME)
#define FILESYSTEM SPIFFS

#define FORMAT_FILESYSTEM false

const int TONE_OUTPUT_PIN = 18;
const int TONE_PWM_CHANNEL = 0;

/* The SMTP Session object used for Email sending */
```

```

SMTPSession smtp;
/* Callback function to get the Email sending status */
void smtpCallback(SMTP_Status status);

File file;

/*name the file*/
const char filename[] = "/Lung.wav";
const int headerSize = 44;

const char* host = "esp32fs";
WebServer server(80);

void listAllFiles(){

    File root = SPIFFS.open("/");
    File file = root.openNextFile();
    while(file){
        Serial.print("FILE: ");
        Serial.println(file.name());
        file = root.openNextFile();
    }

}

//format bytes
String formatBytes(size_t bytes) {
    if (bytes < 1024) {
        return String(bytes) + "B";
    } else if (bytes < (1024 * 1024)) {
        return String(bytes / 1024.0) + "KB";
    } else if (bytes < (1024 * 1024 * 1024)) {
        return String(bytes / 1024.0 / 1024.0) + "MB";
    } else {
        return String(bytes / 1024.0 / 1024.0 / 1024.0) + "GB";
    }
}

String getContentType(String filename) {
    if (server.hasArg("download")) {
        return "application/octet-stream";
    } else if (filename.endsWith(".htm")) {
        return "text/html";
    } else if (filename.endsWith(".html")) {
        return "text/html";
    } else if (filename.endsWith(".css")) {
        return "text/css";
    } else if (filename.endsWith(".js")) {
        return "application/javascript";
    } else if (filename.endsWith(".png")) {
        return "image/png";
    } else if (filename.endsWith(".gif")) {
        return "image/gif";
    } else if (filename.endsWith(".jpg")) {
        return "image/jpeg";
    }
}

```

```

    } else if (filename.endsWith(".ico")) {
        return "image/x-icon";
    } else if (filename.endsWith(".xml")) {
        return "text/xml";
    } else if (filename.endsWith(".pdf")) {
        return "application/x-pdf";
    } else if (filename.endsWith(".zip")) {
        return "application/x-zip";
    } else if (filename.endsWith(".gz")) {
        return "application/x-gzip";
    }
    return "text/plain";
}

bool exists(String path){
    bool yes = false;
    File file = FILESYSTEM.open(path, "r");
    if(!file.isDirectory()){
        yes = true;
    }
    file.close();
    return yes;
}

bool handleFileRead(String path) {
    Serial.println("handleFileRead: " + path);
    if (path.endsWith("/")) {
        path += "index.htm";
    }
    String contentType = getContentType(path);
    String pathWithGz = path + ".gz";
    if (exists(pathWithGz) || exists(path)) {
        if (exists(pathWithGz)) {
            path += ".gz";
        }
        File file = FILESYSTEM.open(path, "r");
        server.streamFile(file, contentType);
        file.close();
        return true;
    }
    return false;
}

void handleFileUpload() {
    if (server.uri() != "/edit") {
        return;
    }
    HTTPUpload& upload = server.upload();
    if (upload.status == UPLOAD_FILE_START) {
        String filename = upload.filename;
        if (!filename.startsWith("/")) {
            filename = "/" + filename;
        }
        Serial.print("handleFileUpload Name: "); Serial.println(filename);
        file = FILESYSTEM.open(filename, "w");
    }
}

```

```

    filename = String();
} else if (upload.status == UPLOAD_FILE_WRITE) {
    //Serial.print("handleFileUpload Data: "); Serial.println(upload.currentSize);
    if (file) {
        file.write(upload.buf, upload.currentSize);
    }
} else if (upload.status == UPLOAD_FILE_END) {
    if (file) {
        file.close();
    }
    Serial.print ("handleFileUpload Size: "); Serial.println(upload.totalSize);
}
}

void handleFileDelete() {
    if (server.args() == 0) {
        return server.send(500, "text/plain", "BAD ARGS");
    }
    String path = server.arg(0);
    Serial.println ("handleFileDelete: " + path);
    if (path == "/" ) {
        return server.send(500, "text/plain", "BAD PATH");
    }
    if (!exists(path)) {
        return server.send(404, "text/plain", "FileNotFound");
    }
    FILESYSTEM.remove(path);
    server.send(200, "text/plain", "");
    path = String();
}

void handleFileCreate() {
    if (server.args() == 0) {
        return server.send(500, "text/plain", "BAD ARGS");
    }
    String path = server.arg(0);
    Serial.println ("handleFileCreate: " + path);
    if (path == "/" ) {
        return server.send(500, "text/plain", "BAD PATH");
    }
    if (exists(path)) {
        return server.send(500, "text/plain", "FILE EXISTS");
    }
    File file = FILESYSTEM.open(path, "w");
    if (file) {
        file.close();
    } else {
        return server.send(500, "text/plain", "CREATE FAILED");
    }
    server.send(200, "text/plain", "");
    path = String();
}

void handleFileList() {
    if (!server.hasArg("dir")) {

```



```

server.send(500, "text/plain", "BAD ARGS");
return;
}
String path = server.arg("dir");
Serial.println("handleFileList: " + path);

File root = FILESYSTEM.open(path);
path = String();

String output = "[";
if(root.isDirectory()){
    File file = root.openNextFile();
    while(file){
        if (output != "[") {
            output += ',';
        }
        output += "{" + "type\":" + "\"";
        output += (file.isDirectory()) ? "dir" : "file";
        output += "\",\"name\":" + "\"";
        output += String(file.name()).substring(1);
        output += "\"}";
        file = root.openNextFile();
    }
}
output += "]";
server.send(200, "text/json", output);
}

/*SPIFFS Initialisation*/
void SPIFFSInit(){
    if(!SPIFFS.begin(true)){
        Serial.println("SPIFFS initialisation failed!");
        while(1) yield();
    }

    SPIFFS.remove(filename);
    file = SPIFFS.open(filename, FILE_WRITE);
    if(!file){
        Serial.println("File is not available!");
    }

    byte header[headerSize];
    wavHeader(header, FLASH_RECORD_SIZE);

    file.write(header, headerSize);
    listSPIFFS();
}

void listSPIFFS(void) {
    Serial.println(F("\r\nListing SPIFFS files:"));
    static const char line[] PROGMEM =
    "=====";

    Serial.println(FPSTR(line));
    Serial.println(F(" File name          Size"));

```



```

Serial.println(FPSTR(line));

fs::File root = SPIFFS.open("/");
if (!root) {
    Serial.println(F("Failed to open directory"));
    return;
}
if (!root.isDirectory()) {
    Serial.println(F("Not a directory"));
    return;
}
fs::File file = root.openNextFile();
while (file) {
    if (file.isDirectory()) {
        Serial.print("DIR: ");
        String fileName = file.name();
        Serial.print(fileName);
    } else {
        String fileName = file.name();
        Serial.print(" " + fileName);
        // File path can be 31 characters maximum in SPIFFS
        int spaces = 33 - fileName.length(); // Tabulate nicely
        if (spaces < 1) spaces = 1;
        while (spaces--) Serial.print(" ");
        String fileSize = (String) file.size();
        spaces = 10 - fileSize.length(); // Tabulate nicely
        if (spaces < 1) spaces = 1;
        while (spaces--) Serial.print(" ");
        Serial.println(fileSize + " bytes");
    }
    file = root.openNextFile();
}
Serial.println(FPSTR(line));
Serial.println();
delay(1000);
}

/*Install the microphone driver and set the communication pin*/
void i2sInit() {
    i2s_config_t i2s_config = {
        .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
        .sample_rate = I2S_SAMPLE_RATE,
        .bits_per_sample = i2s_bits_per_sample_t(I2S_SAMPLE_BITS),
        .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
        .communication_format = i2s_comm_format_t(I2S_COMM_FORMAT_I2S |
I2S_COMM_FORMAT_I2S_MSB),
        .intr_alloc_flags = 0,
        .dma_buf_count = 64,
        .dma_buf_len = 1024,
        .use_apll = 1
    };

    i2s_driver_install(I2S_PORT, &i2s_config, 0, NULL);
    const i2s_pin_config_t pin_config = {
        .bck_io_num = I2S_SCK,

```

```

        .ws_io_num = I2S_WS,
        .data_out_num = -1,
        .data_in_num = I2S_SD
    };
    i2s_set_pin(I2S_PORT, &pin_config);
}

void i2s_adc_data_scale(uint8_t * d_buff, uint8_t* s_buff, uint32_t len)
{
    uint32_t j = 0;
    uint32_t dac_value = 0;
    for (int i = 0; i < len; i += 2) {
        dac_value = (((uint16_t) (s_buff[i + 1] & 0xf) << 8) |
((s_buff[i + 0]))));
        d_buff[j++] = 0;
        d_buff[j++] = dac_value * 256 / 2048;
    }
}

/*start recording*/
void i2s_adc(void *arg)
{
    /*first long buzzer sound signal*/
    ledcAttachPin(TONE_OUTPUT_PIN, TONE_PWM_CHANNEL);
    ledcWriteNote(TONE_PWM_CHANNEL, NOTE_C, 5);
    delay(2000);
    ledcDetachPin(TONE_OUTPUT_PIN);
    delay(5000);

    int i2s_read_len = I2S_READ_LEN;
    int flash_wr_size = 0;
    size_t bytes_read;

    char* i2s_read_buff = (char*) calloc(i2s_read_len, sizeof(char));
    uint8_t* flash_write_buff = (uint8_t*) calloc(i2s_read_len,
sizeof(char));

    i2s_read(I2S_PORT, (void*) i2s_read_buff, i2s_read_len, &bytes_read,
portMAX_DELAY);
    i2s_read(I2S_PORT, (void*) i2s_read_buff, i2s_read_len, &bytes_read,
portMAX_DELAY);

    Serial.println("*** Recording Start ***");
    while (flash_wr_size < FLASH_RECORD_SIZE) {
        //read data from I2S bus, in this case, from ADC.
        i2s_read(I2S_PORT, (void*) i2s_read_buff, i2s_read_len,
&bytes_read, portMAX_DELAY);
        //example_disp_buf((uint8_t*) i2s_read_buff, 64);
        //save original data from I2S(ADC) into flash.
        i2s_adc_data_scale(flash_write_buff, (uint8_t*)i2s_read_buff,
i2s_read_len);
        file.write((const byte*) flash_write_buff, i2s_read_len) //write the
file in flash
        flash_wr_size += i2s_read_len;
        ets_printf("Sound recording %u%%\n", flash_wr_size * 100 /
FLASH_RECORD_SIZE);
    }
}

```

```

        ets_printf("Never Used Stack Size: %u\n",
uxTaskGetStackHighWaterMark(NULL));
    }

    /*second long buzzer sound signal the end of recording*/
    ledcAttachPin(TONE_OUTPUT_PIN, TONE_PWM_CHANNEL);
    ledcWriteNote(TONE_PWM_CHANNEL, NOTE_C, 5);
    delay(2000);
    ledcDetachPin(TONE_OUTPUT_PIN);

    file.close();
    free(i2s_read_buff);
    i2s_read_buff = NULL;
    free(flash_write_buff);
    flash_write_buff = NULL;
    listSPIFFS();
    vTaskDelete(NULL);
}

void example_disp_buf(uint8_t* buf, int length)
{
    printf("=====\n");
    for (int i = 0; i < length; i++) {
        printf("%02x", buf[i]);
        if ((i + 1) % 8 == 0) {
            printf("\n");
        }
    }
    printf("=====\n");
}

/*Header for WAV. soundfile format*/
void wavHeader(byte* header, int wavSize){
    header[0] = 'R'; header[1] = 'I'; header[2] = 'F'; header[3] = 'F';
    unsigned int fileSize = wavSize + headerSize - 8;
    header[4] = (byte)(fileSize & 0xFF); header[5] = (byte)((fileSize >>
8) & 0xFF);
    header[6] = (byte)((fileSize >> 16) & 0xFF); header[7] =
(byte)((fileSize >> 24) & 0xFF);
    header[8] = 'W'; header[9] = 'A'; header[10] = 'V'; header[11] = 'E';
    header[12] = 'f'; header[13] = 'm'; header[14] = 't'; header[15] = '';
    header[16] = 0x10; header[17] = 0x00; header[18] = 0x00; header[19]
= 0x00;
    header[20] = 0x01; header[21] = 0x00; header[22] = 0x01; header[23]
= 0x00;
    header[24] = 0x80; header[25] = 0x3E; header[26] = 0x00; header[27]
= 0x00;
    header[28] = 0x00; header[29] = 0x7D; header[30] = 0x00; header[31]
= 0x00;
    header[32] = 0x02; header[33] = 0x00; header[34] = 0x10; header[35]
= 0x00;
    header[36] = 'd'; header[37] = 'a'; header[38] = 't'; header[39] = 'a';
    header[40] = (byte)(wavSize & 0xFF); header[41] = (byte)((wavSize >>
8) & 0xFF);

```

```

header[42] = (byte)((wavSize >> 16) & 0xFF); header[43] =
(byte)((wavSize >> 24) & 0xFF);
}

void setup() {
  ledcAttachPin(TONE_OUTPUT_PIN, TONE_PWM_CHANNEL);

  WiFi.mode(WIFI_STA);
  Serial.begin(115200);

  WiFiManager wm;
  wm.resetSettings();

  bool res;

  /*set up device AP name and password*/
  res = wm.autoConnect("TeleStethoscope_1", "12345678");

  if(!res) {
    Serial.println("Failed to connect");
    /*three short buzzer sound signal failed to connect Wifi*/
    ledcAttachPin(TONE_OUTPUT_PIN, TONE_PWM_CHANNEL);
    ledcWriteNote(TONE_PWM_CHANNEL, NOTE_G, 5);
    delay(200);
    ledcDetachPin(TONE_OUTPUT_PIN);
    delay(200);
    ledcAttachPin(TONE_OUTPUT_PIN, TONE_PWM_CHANNEL);
    ledcWriteNote(TONE_PWM_CHANNEL, NOTE_G, 5);
    delay(200);
    ledcDetachPin(TONE_OUTPUT_PIN);
    delay(200);
    ledcAttachPin(TONE_OUTPUT_PIN, TONE_PWM_CHANNEL);
    ledcWriteNote(TONE_PWM_CHANNEL, NOTE_G, 5);
    delay(200);
    ledcDetachPin(TONE_OUTPUT_PIN)
  }
  else {
    //if you get here you have connected to the WiFi
    Serial.println("connected...yeey :)");
  }
  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  Serial.println();

  /*two short buzzer sound signal success WiFi connection*/
  ledcAttachPin(TONE_OUTPUT_PIN, TONE_PWM_CHANNEL);
  ledcWriteNote(TONE_PWM_CHANNEL, NOTE_G, 5);
  delay(200);
  ledcDetachPin(TONE_OUTPUT_PIN);
  delay(200);
  ledcAttachPin(TONE_OUTPUT_PIN, TONE_PWM_CHANNEL);
  ledcWriteNote(TONE_PWM_CHANNEL, NOTE_G, 5);
  delay(200);
  ledcDetachPin(TONE_OUTPUT_PIN);

```

```

SPIFFSInit();
i2sInit();
xTaskCreate(i2s_adc, "i2s_adc", 1024 * 2, NULL, 1, NULL);
delay (60000);
Serial.print("\n");
Serial.setDebugOutput(true);
listAllFiles();
if (FORMAT_FILESYSTEM) FILESYSTEM.format();
FILESYSTEM.begin();
{
    File root = FILESYSTEM.open("/");
    File file = root.openNextFile();
    while(file){
        String fileName = file.name();
        size_t fileSize = file.size();
        Serial.printf("FS File: %s, size: %s\n", fileName.c_str(),
formatBytes(fileSize).c_str());
        file = root.openNextFile();
    }
    Serial.printf("\n");
}

/*start create webserver*/
server.on("/list", HTTP_GET, handleFileList);
//load editor
server.on("/edit", HTTP_GET, []() {
    if (!handleFileRead("/edit.htm")) {
        server.send(404, "text/plain", "FileNotFound");
    }
});
//create file
server.on("/edit", HTTP_PUT, handleFileCreate);
//delete file
server.on("/edit", HTTP_DELETE, handleFileDelete);
server.on("/edit", HTTP_POST, []() {
    server.send(200, "text/plain", "");
}, handleFileUpload);

server.onNotFound([]() {
    if (!handleFileRead(server.uri())) {
        server.send(404, "text/plain", "FileNotFound");
    }
});

server.on("/all", HTTP_GET, []() {
    String json = "{";
    json += "\"heap\":" + String(ESP.getFreeHeap());
    json += ", \"analog\":" + String(analogRead(A0));
    json += ", \"gpio\":" + String((uint32_t)0);
    json += "}";
    server.send(200, "text/json", json);
    json = String();
});
server.begin();

```

```

Serial.println("HTTP server started");
//Button

if (!SPIFFS.begin(true)) {
    Serial.println("An error has occurred while mounting SPIFFS");
}
else{
    Serial.println("SPIFFS mounted successfully");
}

/** Enable the debug via Serial port
 * none debug or 0
 * basic debug or 1
 */
smtp.debug(1);

/* Set the callback function to get the sending results */
smtp.callback(smtpCallback);

/* Declare the session config data */
ESP_Mail_Session session;

/* Set the session config */
session.server.host_name = SMTP_HOST;
session.server.port = SMTP_PORT;
session.login.email = AUTHOR_EMAIL;
session.login.password = AUTHOR_PASSWORD;
session.login.user_domain = "mydomain.net";

/* Declare the message class */
SMTP_Message message;

/* Set the message headers */
message.sender.name = "TeleStethoscope_1";
message.sender.email = AUTHOR_EMAIL;
message.subject = "IP ADDRESS";
message.addRecipient("Ms.Ngoc", USER_EMAIL);

/*Send HTML message*/
String ipAdd = WiFi.localIP().toString() + "/Lung.wav";
message.html.content = ipAdd.c_str();
message.text.charset = "us-ascii";
message.html.transfer_encoding = Content_Transfer_Encoding::enc_7bit;

/* Connect to server with the session config */
if (!smtp.connect(&session))
    return;

/* Start sending Email and close the session */
if (!MailClient.sendMail(&smtp, &message))
    Serial.println("Error sending Email, " + smtp.errorReason());

smtp.debug(1);

/* Set the callback function to get the sending results */
smtp.callback(smtpCallback);

```

```

/* Declare the session config data */

/* Set the session config */
session.server.host_name = SMTP_HOST;
session.server.port = SMTP_PORT;
session.login.email = AUTHOR_EMAIL;
session.login.password = AUTHOR_PASSWORD;
session.login.user_domain = "mydomain.net";

/* Declare the message class */
message.enable.chunking = true;

/* Set the message headers */
message.sender.name = "TeleStethoscope_1";
message.sender.email = AUTHOR_EMAIL;
message.subject = "Ms.Ngoc result";
message.addRecipient("Dr.Hung", RECIPIENT_EMAIL1);
message.addRecipient("Mr.Tuong", RECIPIENT_EMAIL2);

/*Send HTML message*/
String htmlMsg1 = "This message contains attachments: wav file.";
message.html.content = htmlMsg1.c_str();
message.text.charset = "us-ascii";
message.html.transfer_encoding = Content_Transfer_Encoding::enc_7bit;

message.priority =
esp_mail_smtp_priority::esp_mail_smtp_priority_normal;
message.response.notify = esp_mail_smtp_notify_success |
esp_mail_smtp_notify_failure | esp_mail_smtp_notify_delay;

/* The attachment data item */
SMTP_Attachment att;

/** Set the attachment info e.g.
 * filename, MIME type, file path, file storage type,
 * transfer encoding and content-encoding
 */
att.descr.filename = "/Lung.wav";
att.descr.mime = "audio/x-wav"; //binary data
att.file.path = "/Lung.wav";
att.file.storage_type = esp_mail_file_storage_type_flash;
att.descr.transfer_encoding = Content_Transfer_Encoding::enc_base64;

message.addAttachment(att);
/* Set the custom message header */
//message.addHeader("Message-ID: <abcde.fghij@gmail.com>");

/* Connect to server with the session config */
if (!smtp.connect(&session))
    return;

/* Start sending Email and close the session */
if (!MailClient.sendMail(&smtp, &message))
    Serial.println("Error sending Email, " + smtp.errorReason());

/*two short buzzer sound signal the end of sending email*/

```

```

ledcAttachPin(TONE_OUTPUT_PIN, TONE_PWM_CHANNEL);
ledcWriteNote(TONE_PWM_CHANNEL, NOTE_G, 5);
delay(200);
ledcDetachPin(TONE_OUTPUT_PIN);
delay(200);
ledcAttachPin(TONE_OUTPUT_PIN, TONE_PWM_CHANNEL);
ledcWriteNote(TONE_PWM_CHANNEL, NOTE_G, 5);
delay(200);
ledcDetachPin(TONE_OUTPUT_PIN);

}

void loop(void) {
    // put your main code here, to run repeatedly:
    server.handleClient();
    delay(2); //allow the cpu to switch to other tasks
}

void smtpCallback(SMTP_Status status){
    /* Print the current status */
    Serial.println(status.info());

    /* Print the sending result */
    if (status.success()){
        Serial.println("-----");
        ESP_MAIL_PRINTF("Message sent success: %d\n", status.completedCount());
        ESP_MAIL_PRINTF("Message sent failed: %d\n", status.failedCount());
        Serial.println("-----\n");
        struct tm dt;

        for (size_t i = 0; i < smtp.sendingResult.size(); i++){
            /* Get the result item */
            SMTP_Result result = smtp.sendingResult.getItem(i);
            time_t ts = (time_t)result.timestamp;
            localtime_r(&ts, &dt);

            ESP_MAIL_PRINTF("Message No: %d\n", i + 1);
            ESP_MAIL_PRINTF("Status: %s\n", result.completed ? "success" : "failed");
            ESP_MAIL_PRINTF("Date/Time: %d/%d/%d %d:%d:%d\n", dt.tm_year + 1900,
dt.tm_mon + 1, dt.tm_mday, dt.tm_hour, dt.tm_min, dt.tm_sec);
            ESP_MAIL_PRINTF("Recipient: %s\n", result.recipients);
            ESP_MAIL_PRINTF("Subject: %s\n", result.subject);
        }
        Serial.println("-----\n");
    }
}

```

We used six different libraries to complete the task of recording sound, setting up the internet connection for the ESP32, and sending the result via email or IP address. The two libraries <driver/i2s.h> and <SPIFFS.h> read the microphone input and store data on SPIFF flash memory on the ESP32. <WebServer.h> and <ESPmDNS.h> serve the upload of data onto the web-based browser. They are included in the browser default

example for ESP32 Dev Module. Lastly, `<WiFiManager.h>` is the library aids in Wi-Fi configuration while `<ESP_Mail_Client.h>` is responsible for the transmission of email.

There are two pieces of information required to connect to a Wi-Fi network, one is the network SSID (Service Set Identifier), which is your network's name, and the other is the network password. Normally, we code our network login parameters (SSID and password) directly in the ESP32 sketch, meaning that the ESP32 can only connect to one network you have coded before. This is a big disadvantage in preventing commercial product creation since you need to edit the Wi-Fi credentials out of your code if you share or distribute the device.

Therefore, a preferable method would be to use a system that lets the users configure their Wi-Fi credentials after the code has been loaded onto your ESP32. This type of system is known as “Wi-Fi Provisioning”. There are several methods for Wi-Fi provisioning. One of them is a Local Access Point (or Local AP for short) method provided by the WiFiManager library which is best to use with the ESP32. This allows the users to use a Wi-Fi-capable phone, tablet, or computer to set up the ESP32 Wi-Fi credentials. (2)

Our result is a .wav file named “Lung.wav”. Its name is defined in the code. ESP32 chip has multiple memory types. Here we use the SPI Flash File System (SPIFFS) to save the .wav file. After recording the file, we have the FSBrowser example to upload the file to a local webserver and then use other devices, such as phones and computers, to connect to the same Wi-Fi network as the ESP32 and download the .wav file. By this way, the file can be transferred wirelessly but cannot be transferred over large distance as it requires the same Wi-Fi network connection. This makes it unsuitable for the telemedicine model. Thus, we come up with the idea to send a simple email with the results as attachment to both the users and doctors. The method is to use an SMTP server to send the email. Each email provider of the sender email, for example, Gmail, Outlook, Live, has a different SMTP server setting. We have to search for its information and make edit in line 16 and 17 of the code.

Here, we use our personal Gmail to send the result to the recipients. The sender email can be replaced with, for example, the product's official result email. The number of recipients is limitless. For the demonstration, in our code there are two email addresses receiving the wav. file result, namely `RECIPIENT_EMAIL1` and `RECIPIENT_EMAIL2`. All

the addresses will receive the .wav sound file while only the USER_EMAIL receives a web address for downloading file as a back-up plan.

Some users' information that can be edited out of the code are:

- Device's AP name and password
- File name
- Recording duration
- Sender and recipients email address
- Sender and recipients' name
- Email subject and content

2.6. Operating procedure

The operation begins with turning ON the device with the button. If the ESP has been turned ON, the lower chamber will light up.



Figure 9. The device in use

Then, the user will use their phone and connect to the “TeleStethoscope_1” Wi-Fi with the password of “12345678”. After that, they will enter the Wi-Fi credential, that is the name and the password of the network for ESP to connect. When successfully connected to the Wi-Fi, the buzzer will make two small beeps.

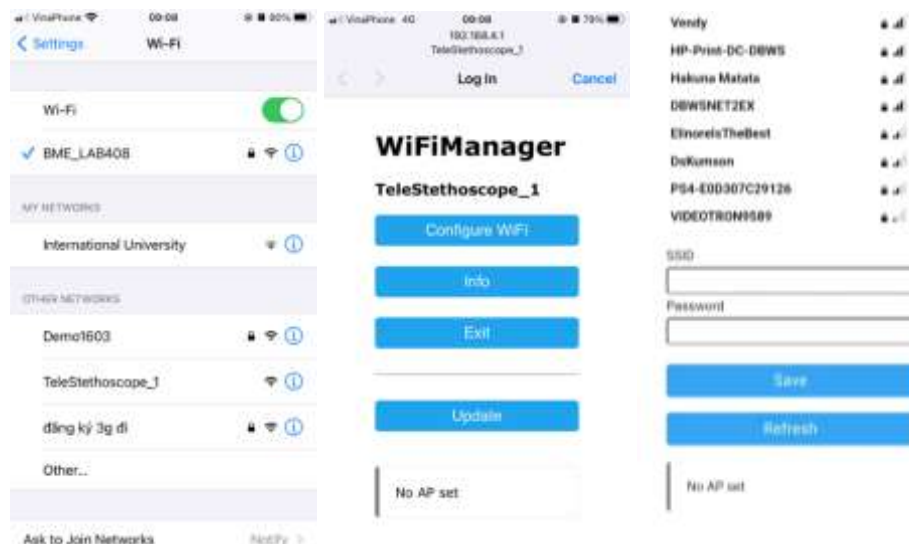


Figure 10. Connecting to Wi-Fi. (a) Choose the AP name “TeleStethoscope_1” and enter the password. (b) Then, the log in server will appear. Click on Configure WiFi button. (c) After scanning for the available networks, the user selects their network SSID and enter the network password.

The recording state will begin after a long beep from the buzzer. After 15 seconds the buzzer will make another long beep, signaling the recording is over. Now the ESP will start transferring the file via email. When this process is done, the buzzer will make two short beeps.

The patient must then check to see if they have received the audio file. If they did, then the device can be turned OFF. If not, then they must keep the device ON while going to the link sent via email to retrieve the audio file via local webserver. After which the device can be turned OFF.

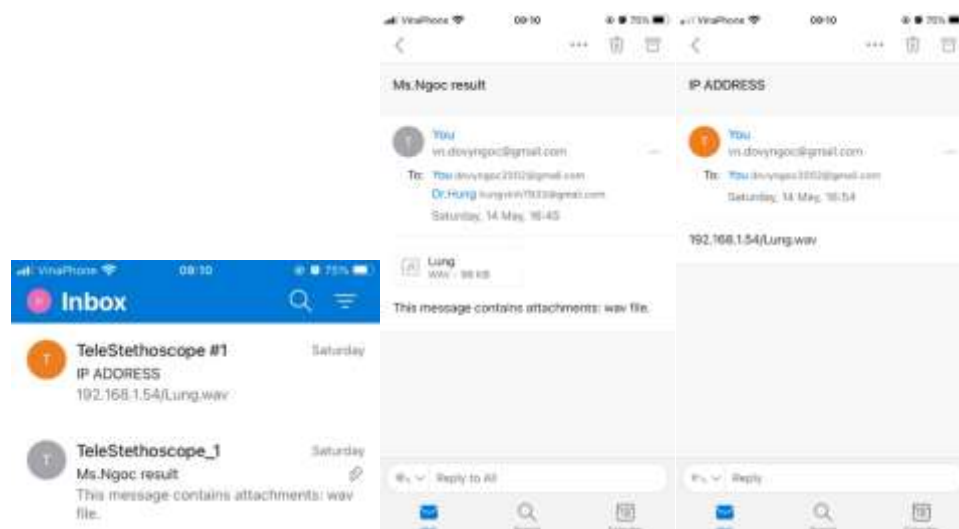


Figure 11. Receiving email. The email subject and content can be edited inside the code. (a) Two mails send to the patient. (b) Doctor only receive the mail with .wav file attachment. (c) Mail with web server address.

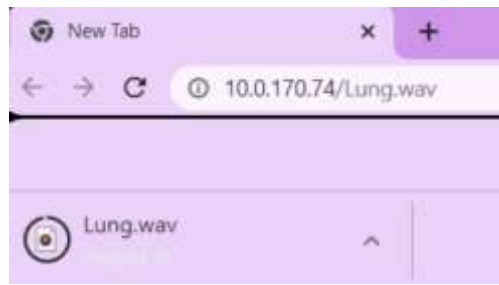


Figure 12. Downloading the files via a local network by copying the address from Figure 12(c).

The full procedure flowchart can be seen below:

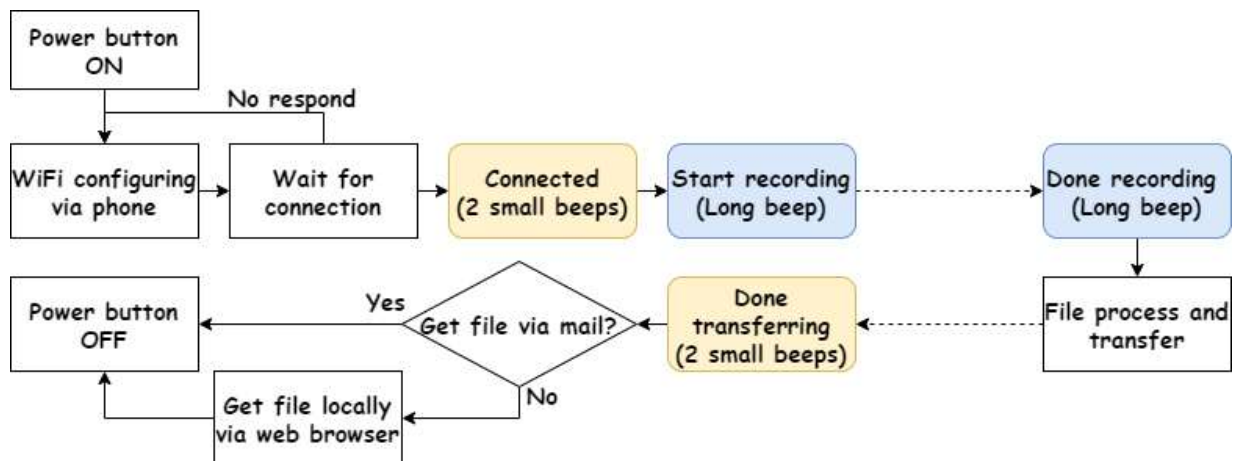


Figure 13. Operating procedure flowchart.

3. Testing Methodology:

To measure the sizes of the product, we would use a caliper and a weight.

The quality of the audio files will be compared with that of a phone microphone. We will first compare a recording of a short conversation (10 seconds) and then the real recording of lung sounds (15 seconds). The aim is to verify that we did record the sound and test the quality of the unfiltered, unprocessed audio.

We have two methods of transferring files, which are either via email or via local webserver. We want to see if either the distance between the transmitter and receiver or the size of the recording can affect this process by trying different distances (1 – 5 – 10 – 20 – 50 meters) and record durations (5 – 10 – 15 – 20 seconds).

The total battery life is measured by fully charging the battery and performing the full recording procedure until the device is no longer working.

4. Results & Discussions:



Figure 14: Telestethoscope's overall appearance

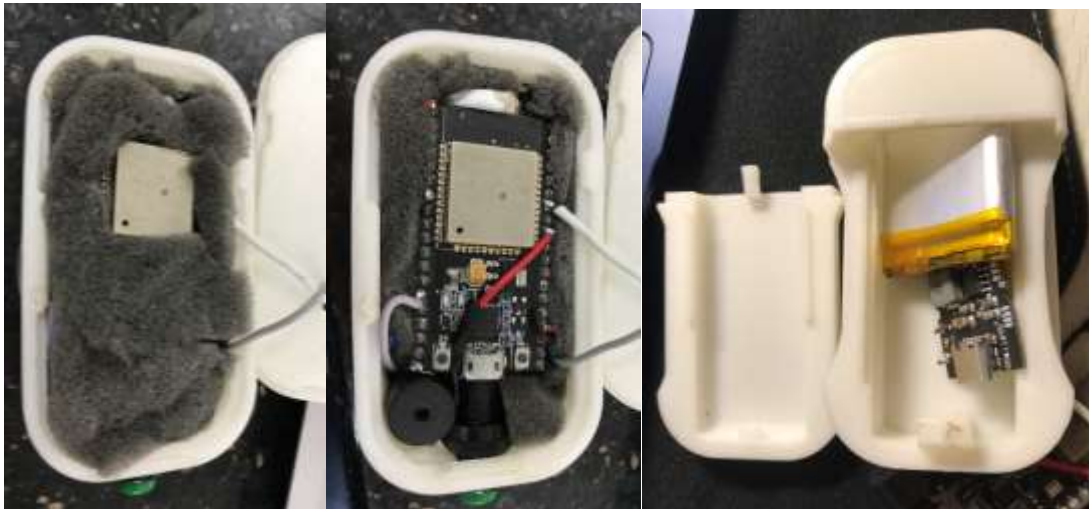


Figure 15: Device's disassembly. From left to right: Main compartment with sound-proofing layer on top, sound-proofing layer removed and battery compartment.



Figure 16: Placement of components inside main chamber. A small rubber earpiece on the left corner connects the stethoscope and microphone.

Our team's product is still not complete, due to many limitations along with some small defects that need to be overcome.

First and foremost, we did achieve some of the initial requirements. The sizes were very small, which measured approximately 45 x 44 x 82 (mm), and the product was indeed portable.



Figure 17: Telestethoscope's dimension.

The operating process only requires a button press and some configuration for the WiFi, so we believe it was simple enough to be applied for patients to use on their own. The buzzer did play a great part in communicating between the user and the ESP but still requires a manual to explain the meaning of different beeps.

As for the audio files, we did record something, as the conversation is still audible from the files. However, from the raw, unprocessed file the heart-lung sound is completely indecipherable with all the noises. Although further processing is possible, not being able to process the sound by itself is still a downside of this product. Upon reviewing the design, we figured out that the recording environment is nowhere perfect, and it allows a lot of rustling sound (between the skin and stethoscope or between the components/wires) to be caught up in the recording.



Figure 18. The plot of the received 15-second .wav file showed the noises from the recording. This is our demonstration of a member's heartbeat.

When transferring files via mail, the distance affects neither the ability nor the speed of upload. However, if the recording is more than 10 seconds long, the upload would very likely fail.

The file size for 15 seconds recording is about 230 KB as the sampling rate and the bits per sample are 16000 and 16 respectively.

Transferring via local webserver works every time but still needs the ESP and the computer to be directly on the same Wi-Fi network. In other words, although the file can be transferred wirelessly, it cannot be transferred over large distance (or from/to different networks), which makes it unsuitable for the telemedicine model.

The battery, once charged fully, can record for 15 seconds duration up to 5 times, which is not very much. However, the battery can be recharged within only 3 minutes, so it is not a big problem.

5. Cost analysis:

Table 3. Payment Table

No.	Product name	Quantity	Price	Actual Spending
1	Li-po charger and boost converter	1	22.000	22.000
2	Lithium polymer battery	1	95.000	95.000
3	INMP441 microphone	1	65.000	130.000
4	ESP32	1	102.000	0
5	Stethoscope	1	50.000	0
6	Buzzer	1	2.000	2.000
7	Led	1	500	0
8	Button	1	3.000	3.000
9	3D printing fees	1	200.000	0

10	Sound-proofing material (10 mm thick)	80 cm ²	1.000	60.000 (for 1 m ² of material)
Total cost			~540.000	312.000

Note: All product prices are calculated in VND.

The final version of our prototype costs approximately 540.000 VND to manufacture but we only spend about 312.000 VND to assemble. The reason for the difference between the estimated price and the actual expenditure is that we were reusing some components from our previous projects. We were not satisfied with the price, but our product would have a long way to go before it could be mass-produced. Until then, it is expected that we can find other solutions, namely replacing components for more reasonable prices or reducing the manufacturing fees.

Although, in comparison to other products on the market, this is a reasonable price. The covid epidemic in several neighboring nations, however, has impeded the logistics of the required materials. According to our analysis, the price of ESP32 has doubled since the provider blocked their border. Other components, such as lithium polymer batteries, are also in short supply. Since these components may come in a variety of versions, acquiring replacements for shattered materials may be difficult. Hopefully, when the epidemic is over, the market will return to normal in terms of price and supply. Another problem contributing to the pricing problem is the high 3D printing fees which accounted for more than one-third of the total cost. We were lucky since our teacher helped us with the first design. However, with other upcoming versions, we must find other ways to lower the price.

6. Conclusion & Limitation & Future works

Upon the completion of the project, our team had the opportunity to apply the theoretical knowledge we have learned to complete a product that can be put into practice. In addition, our team had the opportunity to work together, thus improving our teamwork skills, acquiring more knowledge about operating principles and how a product is completed, and how to manage the said project. Overall, we have achieved the goal of creating a handheld health monitoring product that is functional, convenient, and affordable for the patient. Although the device is incomplete, we are undeniably pleased with what we have achieved for this final 2B project.

On the other hand, it is far from a usable device. The data is fruitless without further processing; and although the data can be transmitted wirelessly, long-range transmission still needs extended adjustments; the stability is pathetically low, and the device frequently fails due to several reasons that we have not figured out or solved yet.

Consequently, there is a lot of room to continue working on this project. Aside from solving the current problems like noise filtering and data transferring, we can make an app to make the operating process more visual and easier to understand. We also need a way to change the recipient emails and their content to create better communication between the doctor and patient. In addition, the design can be reworked to better soundproof the recording section or chamber to increase the quality of the sound.

In conclusion, we completed Final Project 2B with a diagnostic product, which monitors a patient's health through the characteristics of his/her cardiopulmonary sounds and returns the audio files as a result. Our team hopes that our product will bring convenience and reduce costs for the diagnosis of heart and lung diseases. If it is possible, we would like to have another opportunity to produce an improved version of this product, which, with other telemedical devices, will help people in the future.

7. Task Division and Timeline:

Task	Time taken (week)													
	March			April				May					June	
	07/03–	14/03–	21/03–	28/03–	04/04–	11/04–	18/04–	25/04–	02/05–	09/05–	16/05–	23/05–	29/05–	05/06–
Proposal idea														
Proposal														
Materials research														
Buying materials														
Complete the 3D sketch														
Prototype														
Code tests + adjustments														
Data processing														
Solder components														
Report														
Presentation														
	Ngọc	Tường	Hưng	All										

Figure 19. Task division table

Throughout the 3 months' progress, our team, though encountering many obstacles, managed to keep the project on track with the deadlines. There was a postponement in the first half of April because we were all

busy with the midterm exam. However, we made amends the following week, and everything was back on track.

8. Acknowledgment:

We also want to thank Mr Bich, Mr. Giang, Mr. Hùng and Mr. Việt for providing us with the necessary knowledge and showing support for our team project. Without your advice, the project would not have been finished. We also want to appreciate Mr.Bich's generosity in printing the 3D design.

Lastly, we want to send out greatest gratitude to Trung Sơn, our friend, for his assistance with the project.

9. References:

1. *How to design snap-fit joints for 3D printing* | Hubs. Hubs.com. Retrieved 25 May 2022, from <https://www.hubs.com/knowledge-base/how-design-snap-fit-joints-3d-printing/>.
2. *ESP32 WiFiManager - Easy WiFi Provisioning*. DroneBot Workshop. Retrieved 25 May 2022, from <https://dronebotworkshop.com/wifimanager/>.
3. *ESP32 Send Emails using SMTP Server: HTML, Text, Attachments (Arduino IDE)* | Random Nerd Tutorials. Random Nerd Tutorials. (2022). Retrieved 25 May 2022, from <https://randomnerdtutorials.com/esp32-send-email-smtp-server-arduino-ide/>.
4. That Project. (2021). *ESP32 | INMP441 | Tutorial - [Part.4] Capturing audio from i2s mic to save WAV file (I2S interface)* [Video]. Retrieved 25 May 2022, from <https://youtu.be/qmruNKeIN-o>.
5. Wang, B., Liu, Y., Wang, Y., Yin, W., Liu, T., Liu, D., ... & Luo, F. (2020). *Characteristics of pulmonary auscultation in patients with 2019 novel coronavirus in China*. *Respiration*, 99(9), 755-763.